

Traffic Sign Recognition on GTSRB

A convolutional neural network trained from scratch, with a leak-free evaluation protocol

Computer Vision — Project Report

Author: Tanmay Kala (tanmayai23)

Repository: github.com/tanmayai23/gtsrb-traffic-sign-recognition

96.82%

TEST ACCURACY

99,019

PARAMETERS

20 min

TRAINING, CPU ONLY

0.67 ms

PER IMAGE

Abstract

This report describes a 43-class traffic sign classifier built on the German Traffic Sign Recognition Benchmark (GTSRB). The network is a three-block convolutional model written from scratch in PyTorch with 99,019 parameters. It reaches **96.82% accuracy** on the official 12,630-image test set after 15 epochs of CPU training, and classifies an image in 0.67 ms.

The main methodological contribution is the evaluation protocol. GTSRB stores roughly 30 consecutive video frames of every physical sign, so its 39,209 training images depict only 1,307 distinct signs. Splitting that data at random places near-identical frames of one sign on both sides of the train/validation boundary, which lets a model score highly by recognising pictures it has already seen. This project splits by *track* instead, so no physical sign appears on both sides. The resulting accuracy is lower than figures commonly quoted for GTSRB, and it is the number that actually reflects generalisation.

Three ablations confirm that each design decision contributes: removing contrast equalisation costs 1.62 percentage points, removing class rebalancing costs 0.51, and halving the model width costs 5.39.

Contents

1. Introduction and problem statement
2. Dataset
3. Evaluation protocol: splitting by track
4. Preprocessing and augmentation
5. Model architecture
6. Training procedure
7. Results
8. Error analysis
9. Ablation study
10. Explainability
11. Requirements
12. System architecture and design
13. Design decisions and rationale
14. Implementation and testing
15. Challenges and key learnings
16. Limitations and future work
17. Reproducibility · 18. Conclusion · 19. References

1. Introduction and problem statement

Traffic sign recognition is a component of driver assistance and autonomous driving systems. A vehicle must read signs correctly under motion blur, poor lighting, partial occlusion and varying distance, and it must do so fast enough to act on the result. Errors are not symmetric in cost: confusing a 120 km/h sign with an 80 km/h sign has different consequences from confusing two warning triangles.

This project addresses the classification half of that problem. Given an image cropped to a single sign, the task is to assign one of 43 classes. Detection — locating signs in a full road scene — is out of scope, and Section 11 explains what that means for interpreting the results.

Two constraints shaped the work. First, all training had to run on a CPU, which ruled out large architectures and high input resolutions and made the accuracy-per-minute trade-off explicit. Second, the project had to be reproducible from a command line by someone with no prior context, so every step from downloading data to producing figures is a subcommand of a single entry point.

2. Dataset

GTSRB was introduced by Stallkamp et al. for a classification competition at IJCNN 2011. It contains 39,209 training images and 12,630 test images across 43 classes, cropped from video recorded on German roads. Image sizes range from 15×15 to 250×250 pixels. Each image carries an annotated region of interest marking the sign within the frame.

The class distribution is uneven. The most common class holds 1,800 training images and the rarest holds 180, a ratio of 10.0 to 1. This imbalance reflects real road frequencies — 50 km/h signs are simply more common than "beware of ice" — but left unaddressed it biases a classifier toward the frequent classes. Section 6 describes the sampling strategy used to counter it, and Section 9 measures whether it helped.

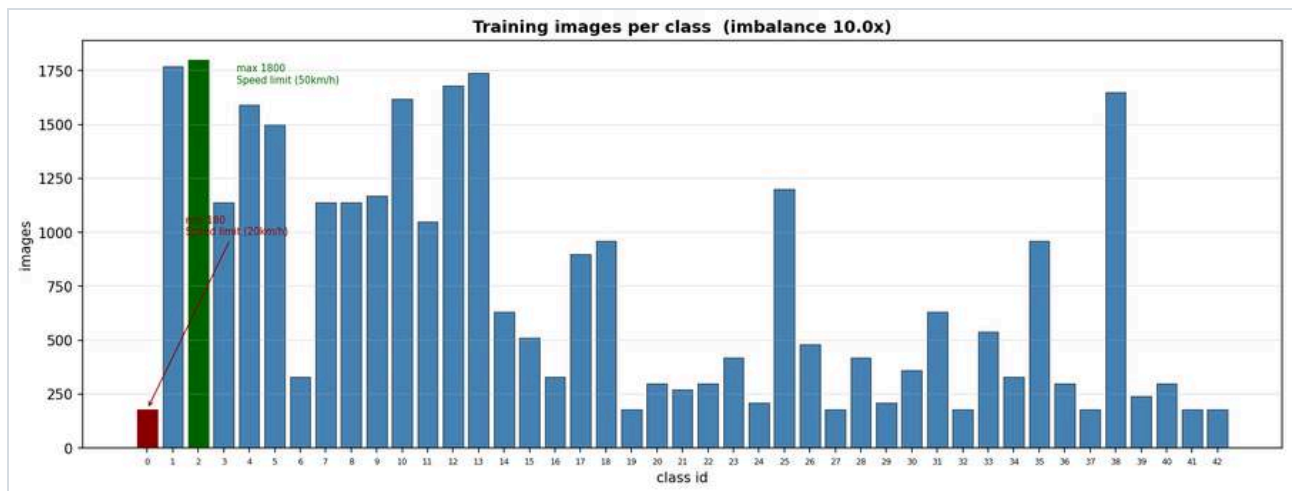


Figure 1. Training images per class. The green bar is the most frequent class and the red bar the rarest. The 10.0× spread is why balanced accuracy is reported alongside raw accuracy throughout.

3. Evaluation protocol: splitting by track

This section describes the most consequential decision in the project.

GTSRB filenames encode two numbers. The file `00053_00024.ppm` is frame 24 of track 53, and a track is a sequence of roughly 30 consecutive video frames of the same physical sign as the camera approaches it. The 39,209 training images therefore depict only 1,307 distinct signs. Frames within a track differ mainly in scale and slight blur; they are near-duplicates.

The consequence for evaluation is direct. A validation split drawn at random over images will place frame 11 of a track in training and frame 12 in validation. The model is then asked to classify a picture almost identical to one it was trained on, and it succeeds — not by generalising, but by recall. Validation accuracy approaches 99.9% and stops carrying information.

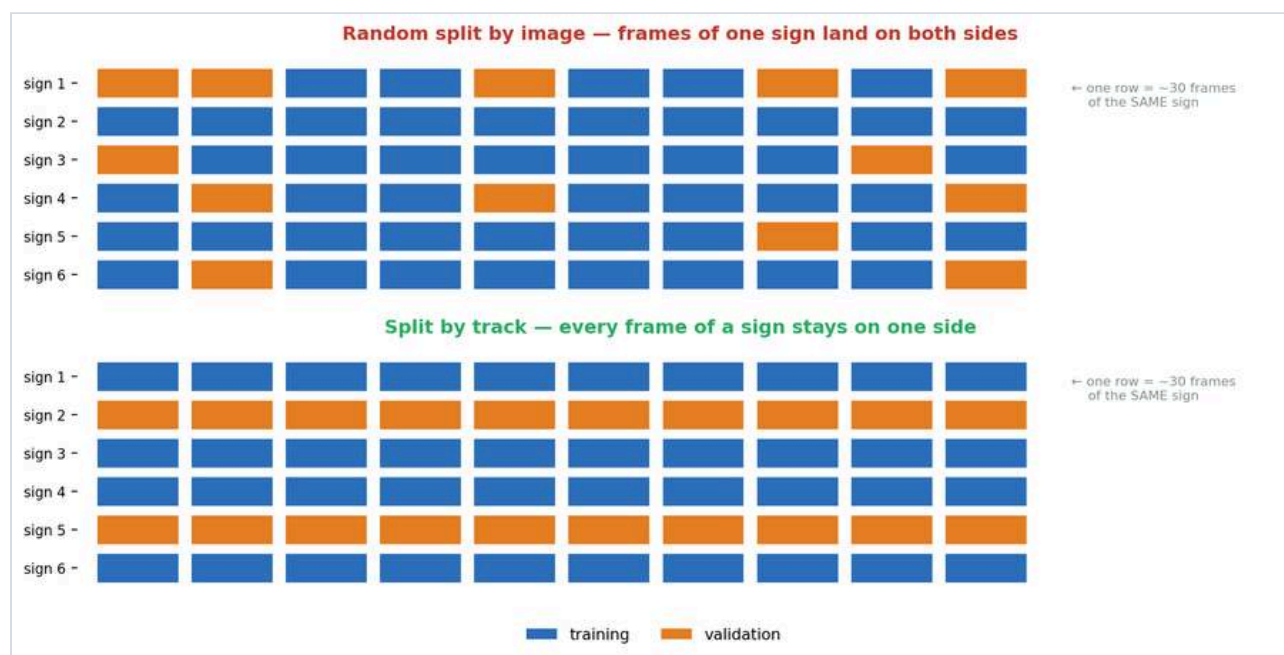


Figure 2. Two ways to split the same data. Each row is one physical sign and each cell one frame. Top: a random split by image scatters frames of a single sign across both sides, so the model is validated on near-duplicates of its training data. Bottom: splitting by track keeps every frame of a sign on one side, so validation images show signs the model has never seen.

This project assigns whole tracks, stratified by class so that every class is represented on both sides. A second detail matters here and is easy to miss: track numbers restart from zero inside each class directory, so track `00000` exists in class 0 and again in class 1 and the two are unrelated. Keying on the bare track number silently merges them. The identifier must combine class and track.

The split is verified rather than assumed. Zero overlap is asserted when the manifests are built, and a unit test suite checks the same properties independently — including one test that confirms a random split *does* leak, documenting the failure being avoided. Those tests run on synthetic data, so the claim can be checked without downloading the dataset.

Table 1. Data splits. Track overlap is zero by construction and by test.

Split	Images	Tracks	Source
Training	31,379	1,046	GTSRB training set
Validation	7,830	261	GTSRB training set, held out by track
Test	12,630	—	Official GTSRB test set, separate recordings
Overlap between train and validation	0	0	asserted and unit-tested

What this costs, and why it is worth paying

Accuracy reported here is lower than the figures near 99% often quoted for GTSRB. Some of that gap is architectural, but some of it is protocol: a model validated on leaked near-duplicates will report a number it cannot reproduce on genuinely new signs. The 96.82% in this report is measured on the official test set, recorded separately from the training data, and the 1.17 point gap between validation (97.98%) and test (96.82%) is small — which is the behaviour a non-leaking split should show.

4. Preprocessing and augmentation

4.1 Preprocessing

Each image passes through four deterministic steps: crop to the annotated region of interest with a 10% margin, apply contrast-limited adaptive histogram equalisation (CLAHE), resize to 32×32 , and normalise using channel statistics computed over the training split only.

The margin on the crop is deliberate. Cropping tight to the annotation removes background clutter, but the outer rim of a sign is itself informative — a red triangle and a blue circle are different classes before any pictogram is read — so a little context is kept.

CLAHE is applied to the L channel in LAB colour space, with a clip limit of 2.0 and a 4×4 tile grid. Two choices are worth stating. Equalising each RGB channel independently shifts hue, and hue carries class information for traffic signs, so the operation is confined to luminance. The tile grid is smaller than OpenCV's 8×8 default because these crops average about 50 pixels per side; the default would leave tiles of roughly six pixels and amplify noise rather than structure. Section 9 shows CLAHE is the single largest preprocessing contributor.

Normalisation statistics are computed over the training split alone (mean [0.414, 0.382, 0.395], standard deviation [0.271, 0.261, 0.269]). Including validation images would leak their distribution into the model's input scaling — a smaller leak than track overlap, but the same kind of mistake.

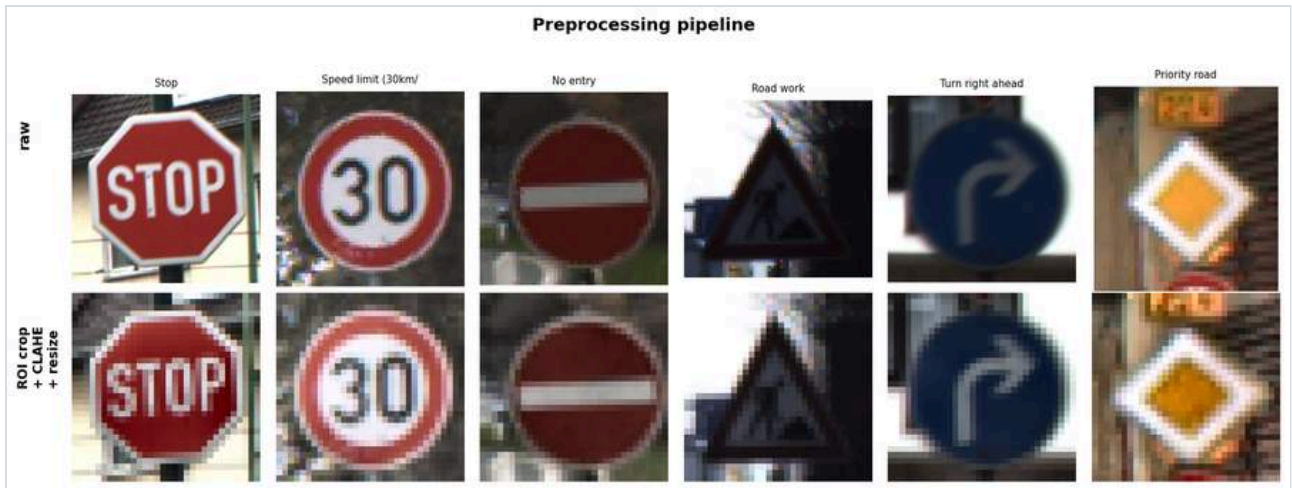


Figure 3. The preprocessing pipeline. Top row: raw images as recorded, with background and uneven exposure. Bottom row: after ROI crop, CLAHE and resize to 32×32 . Contrast is normalised across very different lighting conditions while colour is preserved.

4.2 Augmentation

Training images are augmented with rotation up to $\pm 12^\circ$, translation up to $\pm 10\%$, scaling between 0.9 and 1.1, shear up to ± 0.1 , brightness scaled by 0.7–1.3, contrast by 0.8–1.2, and occasional light Gaussian noise. These ranges mimic the variation a camera actually encounters: a sign viewed from a slightly different angle, distance or light level is still the same sign.

Two implementation details affect quality. All geometric transforms are composed into a single affine matrix and applied in one warp, because chaining three separate warps interpolates three times and visibly degrades a 32-pixel image. Border pixels are replicated rather than filled with black, since black corners would correlate perfectly with "this sample was augmented" and give the network a shortcut unrelated to the sign.

No flips, in any direction

Horizontal flipping is a standard default for natural images and is wrong for this task. Mirroring maps class 33 (turn right ahead) onto class 34 (turn left ahead), class 19 onto class 20 (dangerous curve left and right), class 36 onto class 37, and renders every speed-limit digit unreadable. Flip augmentation would therefore generate training images whose labels are incorrect. The restriction is enforced by a regression test that fails if any augmentation output resembles a mirror of its input more than the input itself.

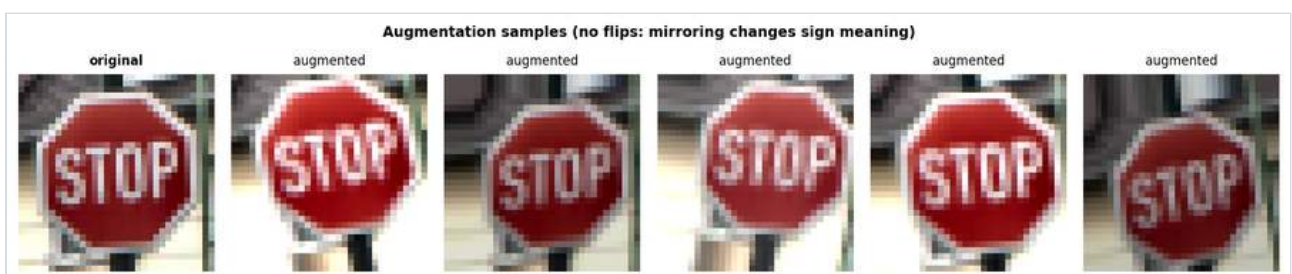


Figure 4. Augmentation samples from one source image. Rotation, scale, shear and brightness vary; the sign is never mirrored, and no black borders are introduced.

5. Model architecture

The network has three convolutional blocks followed by a global-average-pooled linear classifier. Each block is a 3×3 convolution, batch normalisation, ReLU, and 2×2 max pooling, which halves the spatial resolution and doubles the channel count. Total trainable parameters: 99,019.

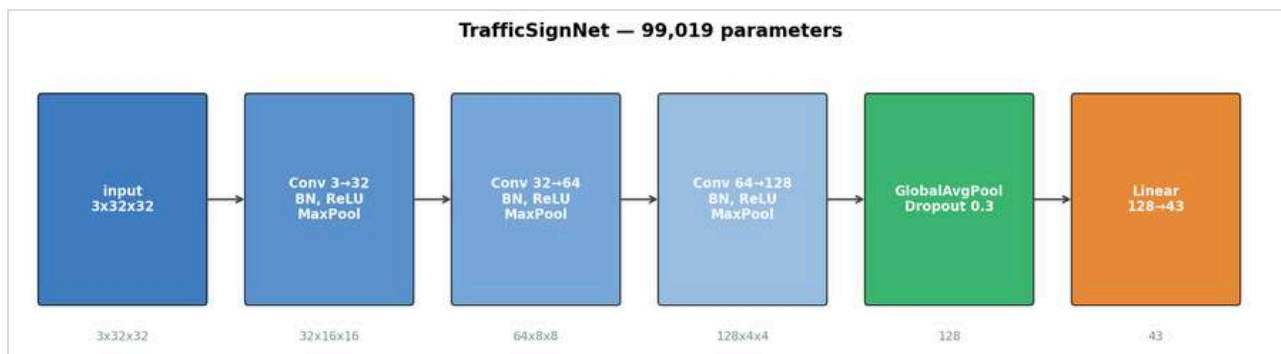


Figure 5. TrafficSignNet. Tensor shapes below each stage. Three pooling operations reduce 32×32 to 4×4 while channels grow from 3 to 128; global average pooling then collapses the spatial dimensions entirely.

Three choices deserve comment. Convolutions omit their bias term because the batch normalisation immediately following re-centres the output, making a convolution bias mathematically redundant. The classifier head uses global average pooling rather than flattening into a fully connected layer: this removes roughly 88,000 parameters, makes the network accept any input resolution without code changes, and is the arrangement under which class activation maps are interpretable (Section 10). Weights are initialised with Kaiming normal scaling, matched to the ReLU activations.

The size of the network was set by measurement rather than preference. On the target hardware, this configuration trains in about 20 minutes. Doubling the input resolution to 48×48 , or using two convolutions per block, each roughly doubled the time per epoch. Section 9 shows that halving the width in the other direction costs 5.39 points of accuracy, so the chosen capacity is not padding.

6. Training procedure

The model is trained with AdamW (learning rate 1×10^{-3} , weight decay 1×10^{-4}) under a OneCycle schedule peaking at 3×10^{-3} , stepped every batch rather than every epoch. The loss is cross-entropy with label smoothing of 0.05, and gradients are clipped at norm 5.0. Class imbalance is handled by a weighted random sampler that draws inversely to class frequency.

OneCycle suits a fixed short budget: the warm-up allows a higher peak learning rate than would otherwise be stable, and the anneal brings the model to a good minimum within about fifteen epochs, where a step schedule would still be descending. Training stops early if validation accuracy fails to improve for six epochs, and a wall-clock guard stops cleanly if a time budget is exceeded. Checkpoints are written every epoch, so an interrupted run is never lost, and each checkpoint stores its own configuration, class names and normalisation statistics so that inference needs no matching flags.

Table 2. Training history (selected epochs of 15). Best validation accuracy occurred at epoch 13.

Epoch	Train loss	Train acc (%)	Val loss	Val acc (%)	Time (s)
1	3.2577	13.42	2.9117	19.18	82
2	2.3458	36.00	2.2634	33.70	84
3	1.7478	56.25	1.7935	55.54	85
5	0.9540	87.28	1.0831	82.12	84
8	0.6849	96.11	0.6959	94.84	78
10	0.6270	97.56	0.6103	97.37	82
12	0.5898	98.30	0.5785	97.87	85
13	0.5788	98.64	0.5683	97.98	85
14	0.5726	98.83	0.5724	97.87	83
15	0.5701	98.86	0.5692	97.84	84

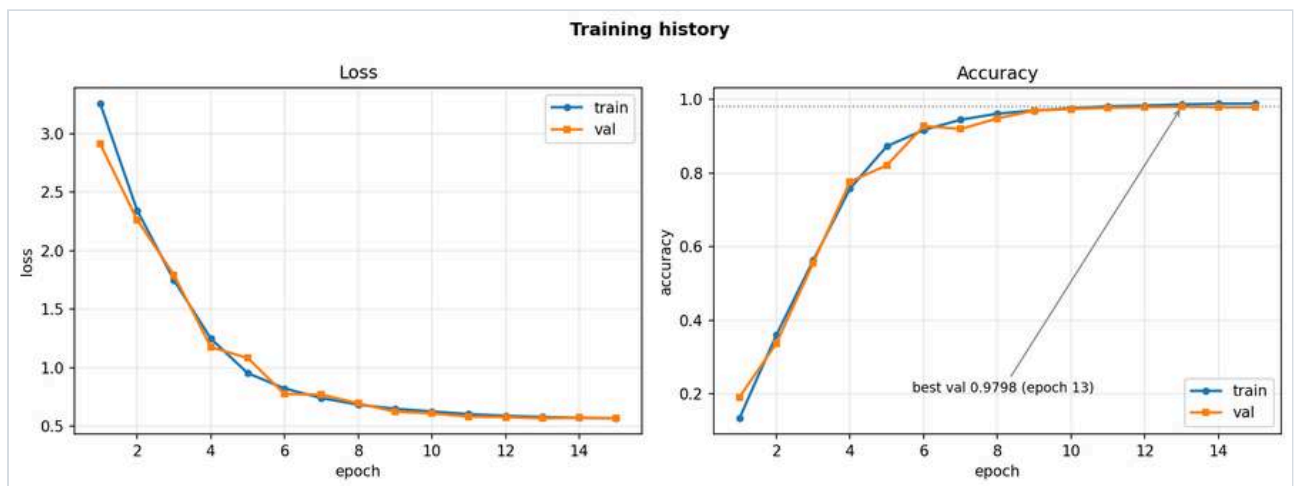


Figure 6. Loss and accuracy per epoch. Training and validation curves stay close throughout, indicating the augmentation and dropout are sufficient to prevent overfitting; there is no widening gap.

The two curves track each other closely for the whole run. Final training accuracy is 98.86% against validation 97.84%, a gap under one and a half points. Combined with the track-disjoint split, this indicates the model is learning sign appearance rather than memorising particular photographs.

7. Results

All figures below are measured on the official GTSRB test set of 12,630 images, which comes from separate recording sessions and is used only once, after training is complete.

Table 3. Test set performance.

Metric	Value	Interpretation
Accuracy	96.82%	12,228 of 12,630 correct
Balanced accuracy	95.82%	mean per-class recall; unaffected by class frequency
Macro F1	95.67%	every class weighted equally
Weighted F1	96.80%	weighted by class frequency
Top-5 accuracy	99.56%	true class within the five highest-scoring
Errors	402	misclassified images
Inference time	0.67 ms	per image, CPU, batch size 256

Balanced accuracy is reported alongside raw accuracy deliberately. Under a 10.0-to-1 imbalance, a model can post a high overall accuracy while performing poorly on rare classes, because the frequent classes dominate the average. The 1.00 point gap between the two here is modest, which indicates the rare classes are not being neglected.

Of the 43 classes, 3 reach a perfect F1 score, 6 exceed 0.99, and 3 fall below 0.90. Top-5 accuracy of 99.56% means that even when the model is wrong, the correct class is almost always among its leading candidates — the errors are confusions between similar signs rather than failures to recognise a sign at all.

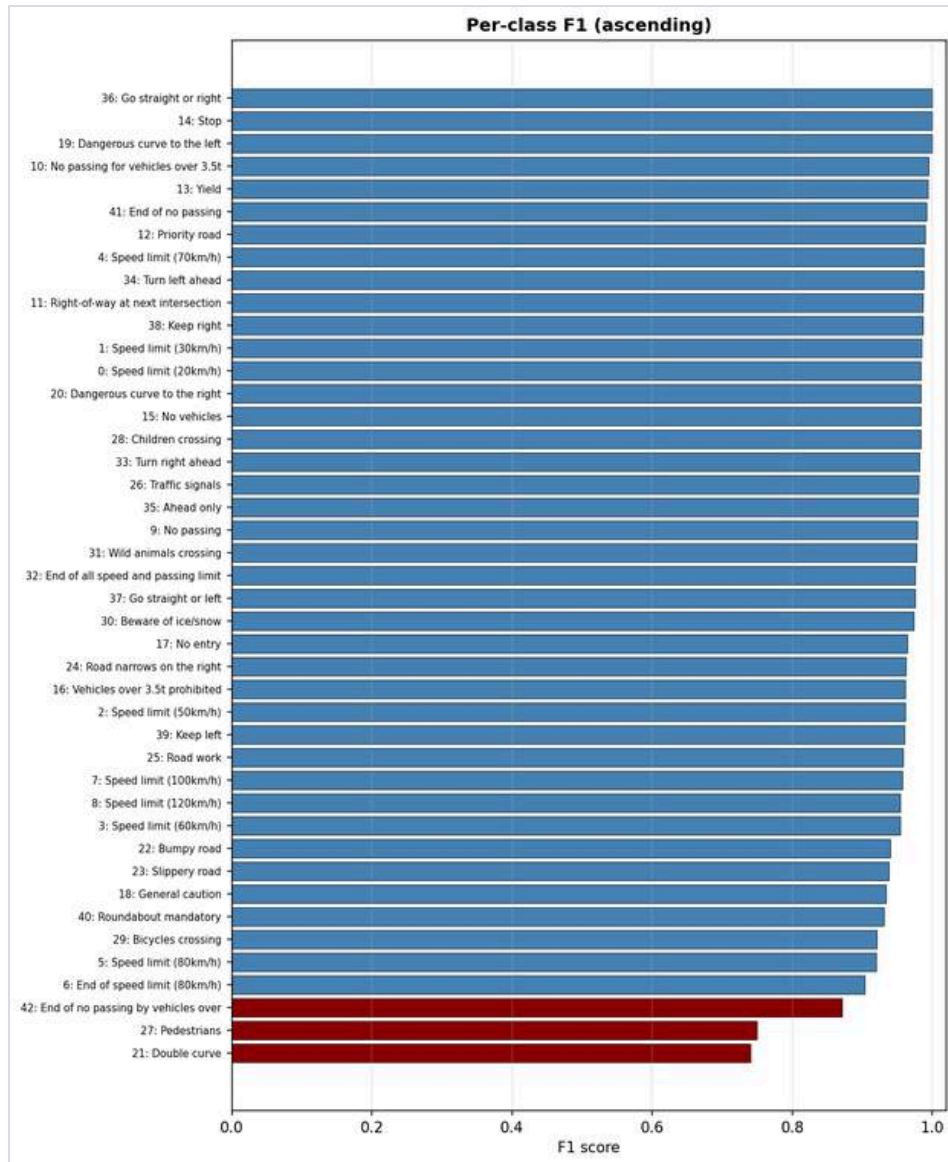


Figure 7. Per-class F1, ascending. Red bars mark classes below 0.90. Performance is high and even across most of the 43 classes, with a short tail of difficult ones analysed in Section 8.

8. Error analysis

The 402 errors are not spread uniformly. They concentrate in groups of signs that are genuinely similar at the working resolution.

Table 4. Ten most frequent confusions. "Share of class" is the proportion of that true class sent to the wrong label.

True	Class	Pred	Predicted as	Count	Share of class
8	Speed limit (120km/h)	5	Speed limit (80km/h)	29	6.4%
21	Double curve	18	General caution	27	30.0%
5	Speed limit (80km/h)	2	Speed limit (50km/h)	23	3.6%
6	End of speed limit (80km/h)	42	End of no passing by vehicles over 3.5t	23	15.3%
3	Speed limit (60km/h)	5	Speed limit (80km/h)	20	4.4%
17	No entry	7	Speed limit (100km/h)	17	4.7%
2	Speed limit (50km/h)	1	Speed limit (30km/h)	12	1.6%
5	Speed limit (80km/h)	7	Speed limit (100km/h)	12	1.9%
27	Pedestrians	23	Slippery road	12	20.0%
3	Speed limit (60km/h)	2	Speed limit (50km/h)	11	2.4%

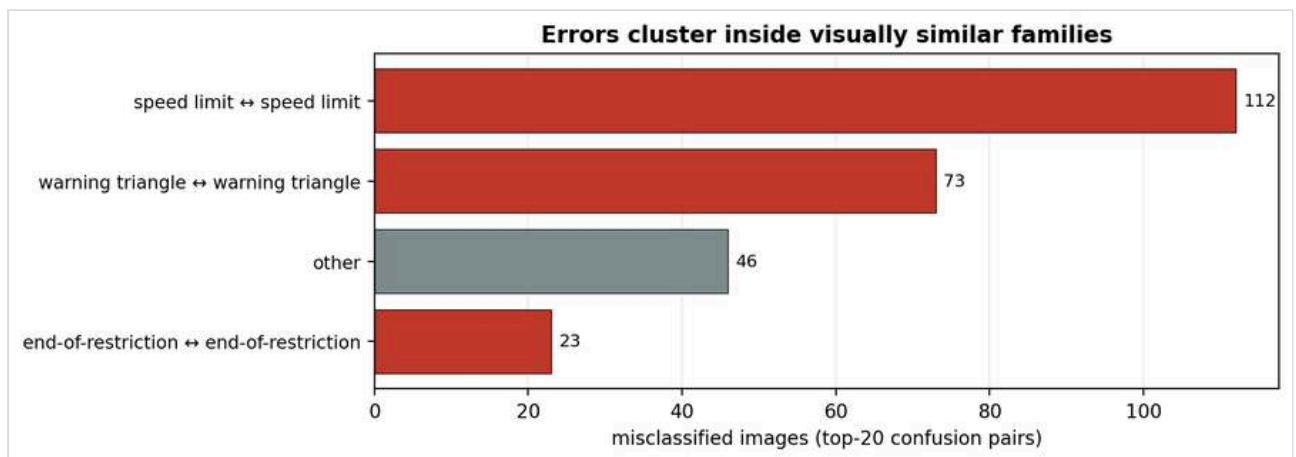


Figure 8. Errors from the twenty most frequent confusion pairs, grouped by sign family. 208 of 254 (82%) are confusions within a visually similar family rather than between unrelated signs.

Two families dominate. Speed-limit signs differ only in one or two digits inside an identical red circle; at 32×32 those digits span a handful of pixels, and 120 becomes hard to separate from 80. Warning triangles share an identical red border and differ only in a small central pictogram, which suffers the same fate. Both patterns point to the same cause — input resolution — and Section 11 identifies raising it as the clearest route to improvement.

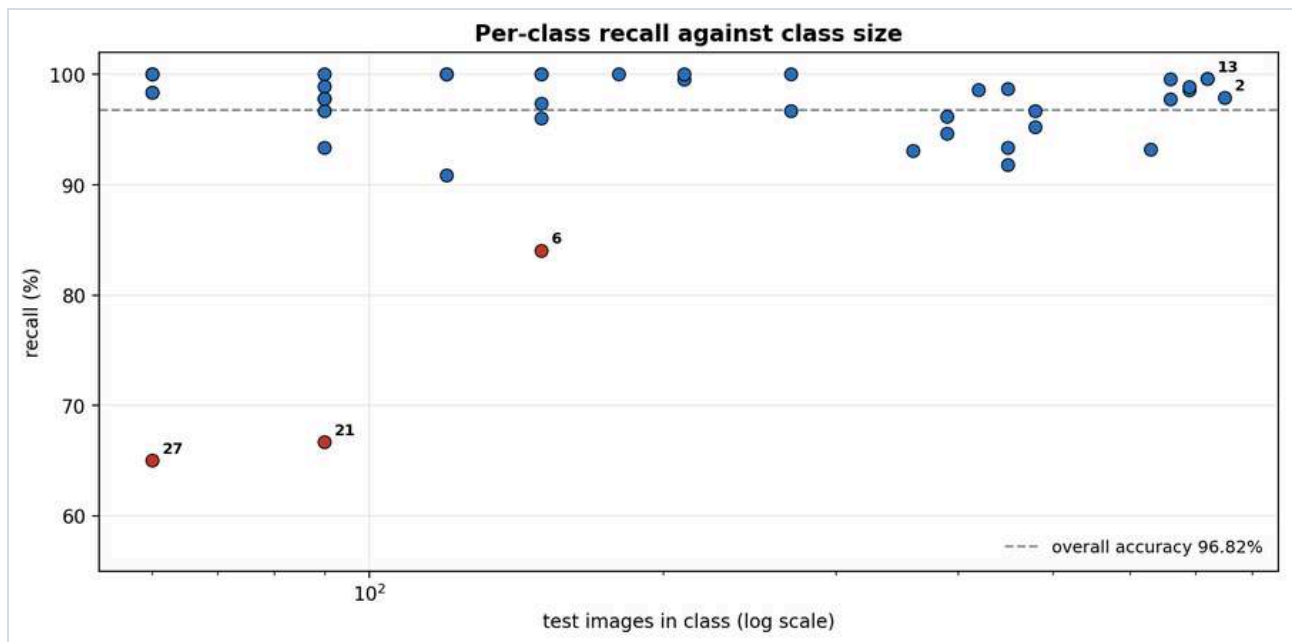


Figure 9. Per-class recall against the number of test images in that class. Red points fall below 0.90 recall. The weakest classes are not the smallest ones, which indicates the difficulty comes from visual similarity rather than lack of training data.

This last figure answers a question the imbalance raises. If the rare classes were simply under-trained, recall would fall as class size fell. It does not: several of the smallest classes reach perfect recall, while the weakest performers sit in the middle of the size range. Class 21 (double curve) and class 27 (pedestrians) are hard because they resemble other warning triangles, not because they are rare. This also explains the modest return from rebalancing measured in Section 9.

Table 5. The six lowest-scoring classes by F1.

ID	Class	Precision	Recall	F1	Support
21	Double curve	0.833	0.667	0.741	90
27	Pedestrians	0.886	0.650	0.750	60
42	End of no passing by vehicles over 3.5t	0.786	0.978	0.871	90
6	End of speed limit (80km/h)	0.977	0.840	0.903	150
5	Speed limit (80km/h)	0.909	0.932	0.920	630
29	Bicycles crossing	0.879	0.967	0.921	90

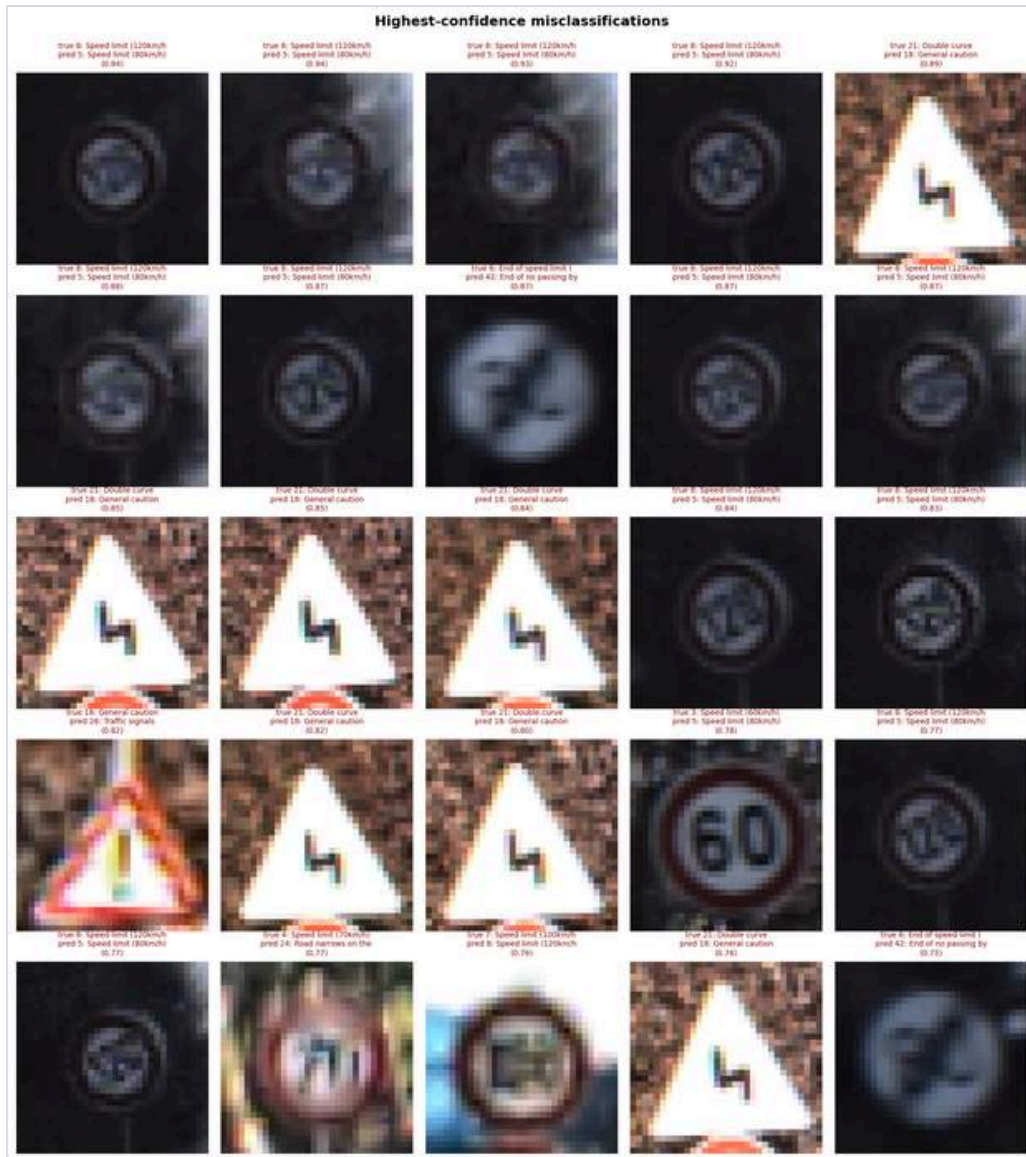


Figure 10. The twenty-five errors the model was most confident about. High-confidence mistakes are more informative than random ones: they reveal systematic confusions rather than unreadable crops.

9. Ablation study

Each row below is a complete retraining run of 15 epochs with one design decision removed, evaluated on the same test set under identical conditions. The purpose is to establish that each choice earns its place rather than being inherited convention.

Table 6. Ablation results. Δ is the change in accuracy against the baseline, in percentage points.

Configuration	Command	Acc (%)	Balanced (%)	Macro F1 (%)	Params	Δ (pp)
Baseline (all decisions active)	<code>train</code>	96.82	95.82	95.67	99,019	—
Without CLAHE	<code>train --no-clahe</code>	95.19	94.03	93.73	99,019	-1.62
Without class rebalancing	<code>train --balance none</code>	96.31	94.67	95.03	99,019	-0.51
Half-width model	<code>train --width-mult 0.5</code>	91.43	92.03	90.73	26,491	-5.39

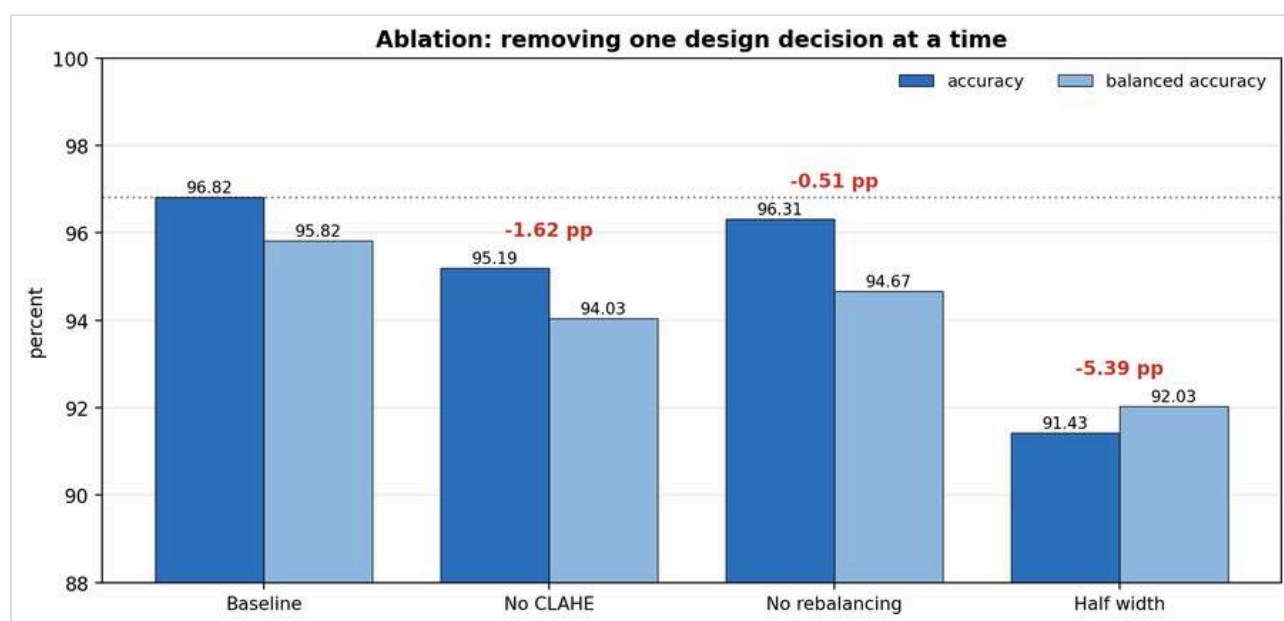


Figure 11. Ablation results. The dotted line marks baseline accuracy. Each removal degrades performance, confirming that all three decisions contribute.

Contrast equalisation matters most (+1.62 points). GTSRB is dashcam footage containing heavily under- and over-exposed frames, and normalising local contrast before the network sees an image is worth more than any other preprocessing step tested.

Rebalancing helps rare classes specifically. Removing it costs 0.51 points of raw accuracy but 1.15 points of balanced accuracy — more than twice as much. That asymmetry is exactly the signature expected from an intervention aimed at the tail of the distribution: it barely moves the average, which frequent classes dominate, while measurably improving the rare classes that balanced accuracy weights equally. The effect is real but modest, consistent with Section 8's finding that difficulty here comes from visual similarity rather than class size.

The chosen capacity is justified. Halving the width reduces the model to 26,491 parameters and costs 5.39 points, the largest single drop measured. The 99,019-parameter baseline is therefore not oversized for the task.

10. Explainability

Accuracy alone does not establish that a model is using the right evidence. A classifier can reach a high score by keying on background correlations — sky, road texture, the pole a sign is mounted on. Grad-CAM (Selvaraju et al., 2017) tests this by computing which spatial regions of the final convolutional layer drive a particular prediction.

The implementation here is written directly with forward and full-backward hooks and uses no external library. Gradients of the target logit with respect to the final convolutional feature maps are averaged spatially to form per-channel weights; the weighted sum of feature maps is then rectified and normalised. Two details are easy to get wrong: the backward pass must use `register_full_backward_hook`, since the older hook API reports incorrect gradients for multi-input modules, and the forward pass must run with gradients enabled — the model is in evaluation mode for batch normalisation, but wrapping it in `no_grad` or `inference_mode` silently produces an empty map.

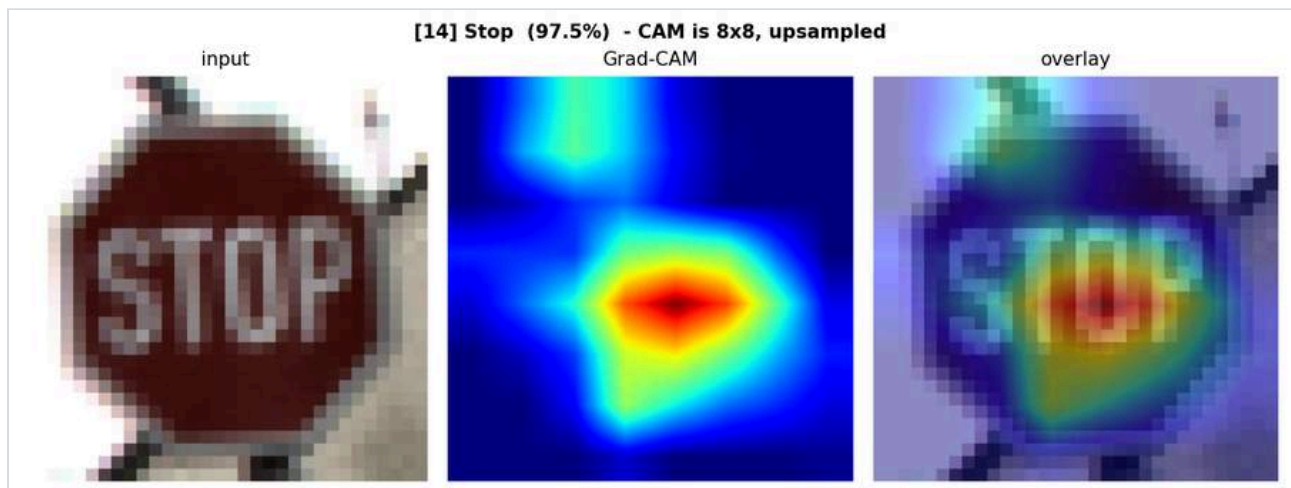


Figure 12. Grad-CAM for a stop sign, classified correctly with 97.5% confidence. Attention concentrates on the sign face and lettering; the background is cold. The map is 8×8 before upsampling, so it is coarse — sufficient to distinguish sign from background, not to localise individual characters.

Across the sample images, activation consistently centres on the sign rather than its surroundings, which supports the conclusion that the network has learned sign appearance. The resolution limit should be stated plainly: at 32×32 input with three pooling stages, the map is 8×8 , so it can show that the model looks at the sign but not which stroke of a digit it relies on.

11. Requirements

11.1 Functional requirements

The system is organised into six functional modules. The first three are the core pipeline; the remaining three support analysis and reporting. Each requirement below is implemented by a named module and checked either by the test suite or by a committed artefact.

Module	Requirements	Implemented in
FM-1 Data preparation	Resumable download and size-checked, zip-slip-safe extraction; parse per-image annotations into <code>(path, class, ROI, track)</code> ; group by the composite key <code>(class_id, track_number)</code> ; emit a track-disjoint, class-stratified split; assert disjointness at build time.	<code>download.py</code> , <code>prepare.py</code>
FM-2 Training	Define the CNN from scratch; apply ROI crop → CLAHE → resize → normalise; augment without ever mirroring; counter class imbalance; compute normalisation statistics from the training split only; train with OneCycle, early stopping and a wall-clock budget; checkpoint the best model with its config.	<code>model.py</code> , <code>transforms.py</code> , <code>dataset.py</code> , <code>engine.py</code> , <code>train.py</code>
FM-3 Evaluation and inference	Evaluate on the official test set; report accuracy, balanced accuracy, macro/weighted F1 and top-5; per-class metrics; confusion matrix and ranked confusion pairs; highest-confidence error grid; single-image prediction with a JSON mode; inference latency.	<code>evaluate.py</code> , <code>predict.py</code> , <code>plots.py</code>
FM-4 Explainability	Grad-CAM over the last convolutional layer for a chosen or predicted class, using full backward hooks that are removed afterwards.	<code>gradcam.py</code>
FM-5 Ablation study	Disable individual design decisions by CLI flag and retrain; write each run's metrics to its own directory for comparison.	<code>cli.py</code> , <code>config.py</code>
FM-6 Reporting and interface	Single CLI entry point with per-command help; environment status; regenerate all result figures and design diagrams from code; rebuild this report from the current metrics.	<code>cli.py</code> , <code>report_figures.py</code> , <code>design_figures.py</code> , <code>build_report.py</code>

Input/output structure. The pipeline is a chain of file-backed stages, which is what makes each stage independently re-runnable: the archives produce extracted images and manifests; the manifests plus hyperparameters produce a checkpoint and a training history; the checkpoint plus the test manifest produce `metrics.json` , CSVs and figures; and a checkpoint plus a single image produces a prediction or a heatmap.

11.2 Non-functional requirements

#	Requirement	How it is met, and the evidence
NFR-1	Reproducibility	All three RNGs seeded from one <code>--seed</code> ; augmentation uses a per-index generator so results hold with parallel loading; the split is a pure function of <code>(seed, val_frac)</code> ; checkpoints carry their own config and normalisation statistics. Caveat in Section 17.
NFR-2	Correctness of measurement	Track-disjoint split asserted in code and tests; the official test set is used only once, for final evaluation; normalisation statistics from training only; imbalance-robust metrics reported beside raw accuracy. Evidence: the validation-to-test gap of 1.17 points.
NFR-3	Resource efficiency	CPU-only; 20 min to train; ~96 MB uint8 RAM cache; 99,019 parameters in a ~1.2 MB checkpoint; 0.67 ms per image. <code>--quick</code> , <code>--epochs</code> , <code>--max-minutes</code> , <code>--batch-size</code> and <code>--img-size</code> trade accuracy for time or memory.
NFR-4	Reliability and error handling	Downloads resume and are size-checked before use; extraction rejects escaping paths; <code>last.pt</code> is written every epoch so an interrupted run is never wasted; the split fails loudly rather than emitting leaking data.
NFR-5	Maintainability	15 single-purpose modules in four layers with no upward dependencies; configuration centralised in <code>config.py</code> ; every figure and diagram generated from code, so documentation cannot silently drift.
NFR-6	Testability	50 tests; the central claims run on synthetic data and pass on a bare clone; dataset-dependent tests skip rather than fail. See Section 14.
NFR-7	Usability	One entry point with discoverable subcommands; a committed checkpoint so inference works immediately; <code>info</code> reports environment and data status; documented troubleshooting for the failures that actually occur.
NFR-8	Portability	Windows, Linux and macOS, with run scripts for PowerShell and bash; pinned dependencies; the headless <code>Agg</code> <code>matplotlib</code> backend; <code>.gitattributes</code> marks <code>.ppm</code> binary so git cannot corrupt sample pixels.

The system processes a public research dataset and exposes no network service, authentication or user data, so conventional security requirements largely do not apply. The two that do — safe archive extraction and size-checked downloads — are treated as reliability requirements above.

12. System architecture and design

The code is arranged in four layers, and dependencies only ever point downward. The CLI parses arguments and delegates; the application layer holds one module per use case; the domain layer holds the data, model and transform logic that the use cases compose; and everything is persisted as plain files rather than a database, because every artefact is either a bulk binary (images, weights) or a small structured document (manifests, metrics) that benefits from being diffable and inspectable without tooling.

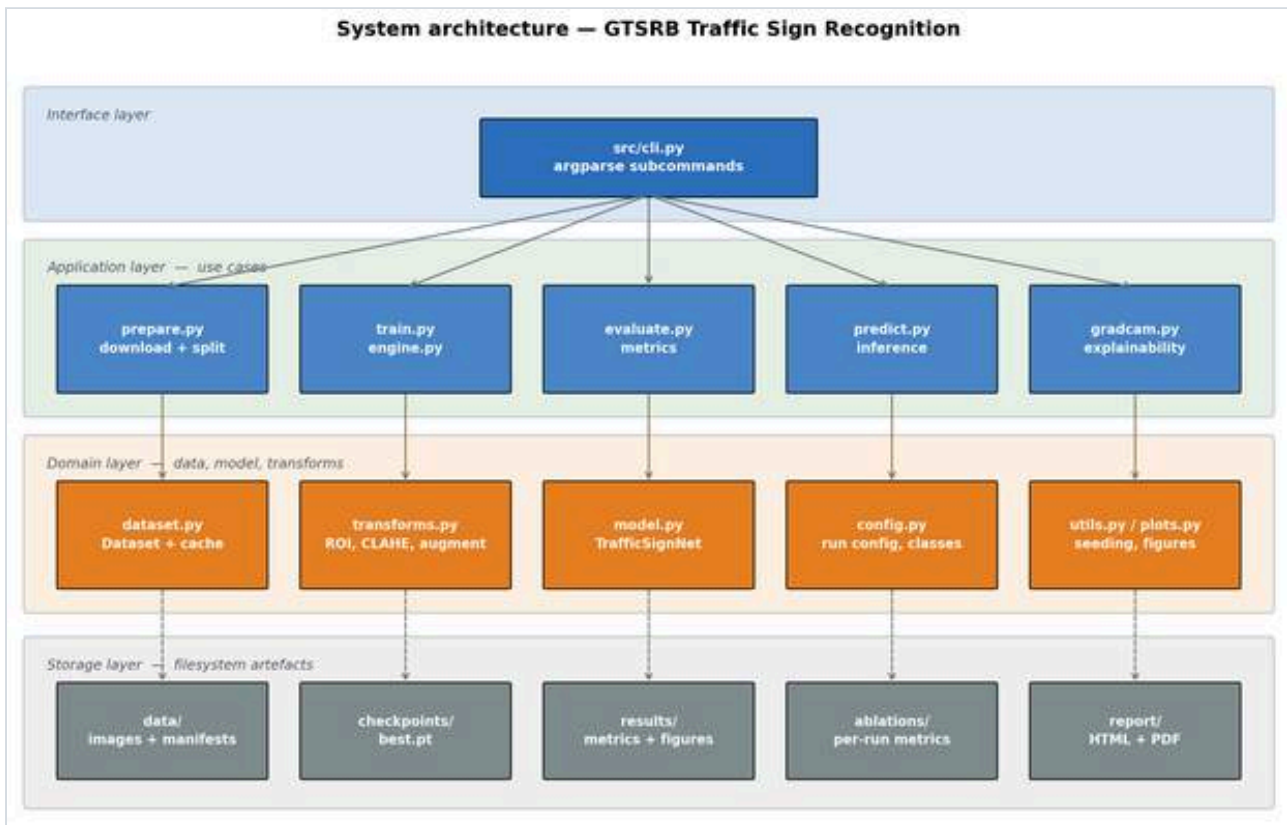


Figure 13. System architecture. Arrows point from a consumer to what it depends on; no module calls upward into a higher layer.

Storage design. There is no relational database, so no ER diagram applies. The equivalent schema is the manifest record, which is the unit that flows through the whole pipeline:

Field	Type	Meaning
path	string	image location, relative to the data root
class_id	int 0–42	ground-truth label
track_id	int	which physical sign — unique only within a class
roi_x1, roi_y1, roi_x2, roi_y2	int	annotated sign bounding box
width, height	int	source image dimensions

The composite `(class_id, track_id)` is the grouping key for the split, and the reason it must be composite is given in Section 3. Alongside the manifests, `split_summary.json` records the resulting counts and `norm_stats.json` the training-split channel statistics; both are read back by later stages rather than recomputed, so a stage cannot disagree with the one before it.

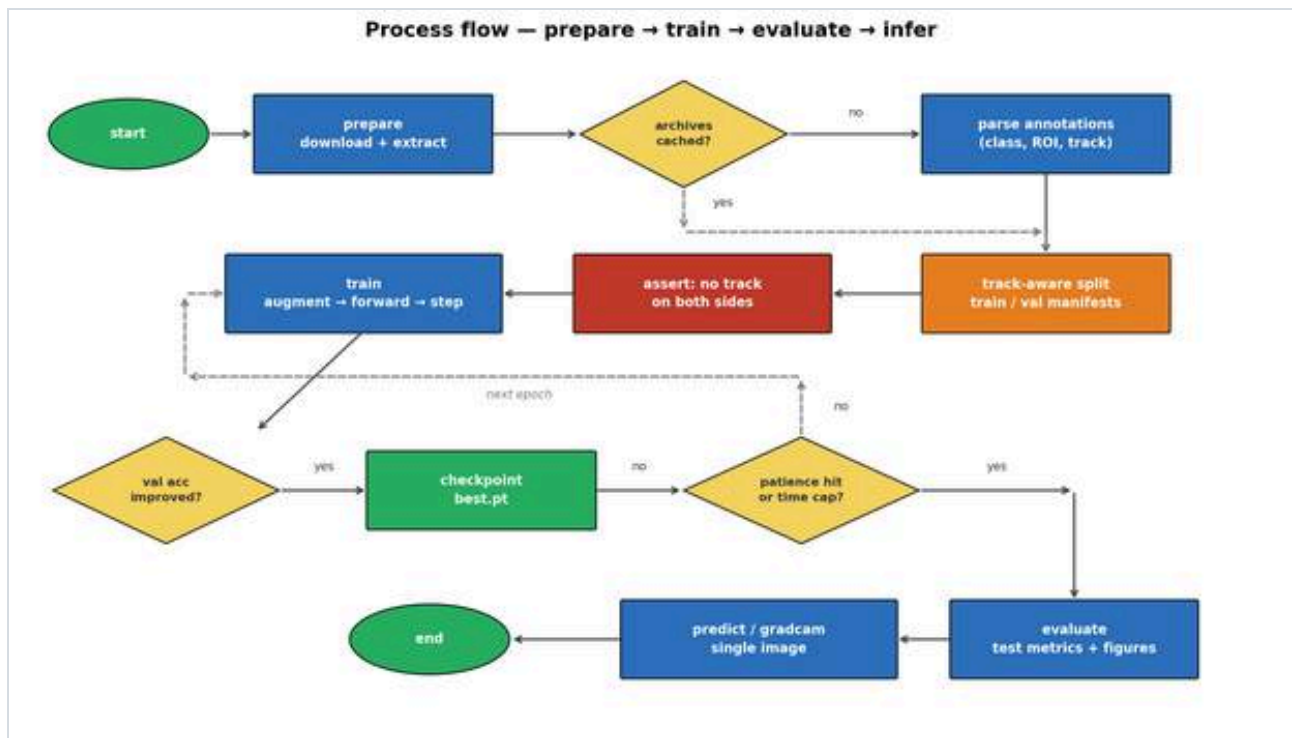


Figure 14. Process flow. Decision points are the two that genuinely branch: whether the archives are already cached, and whether training should continue.

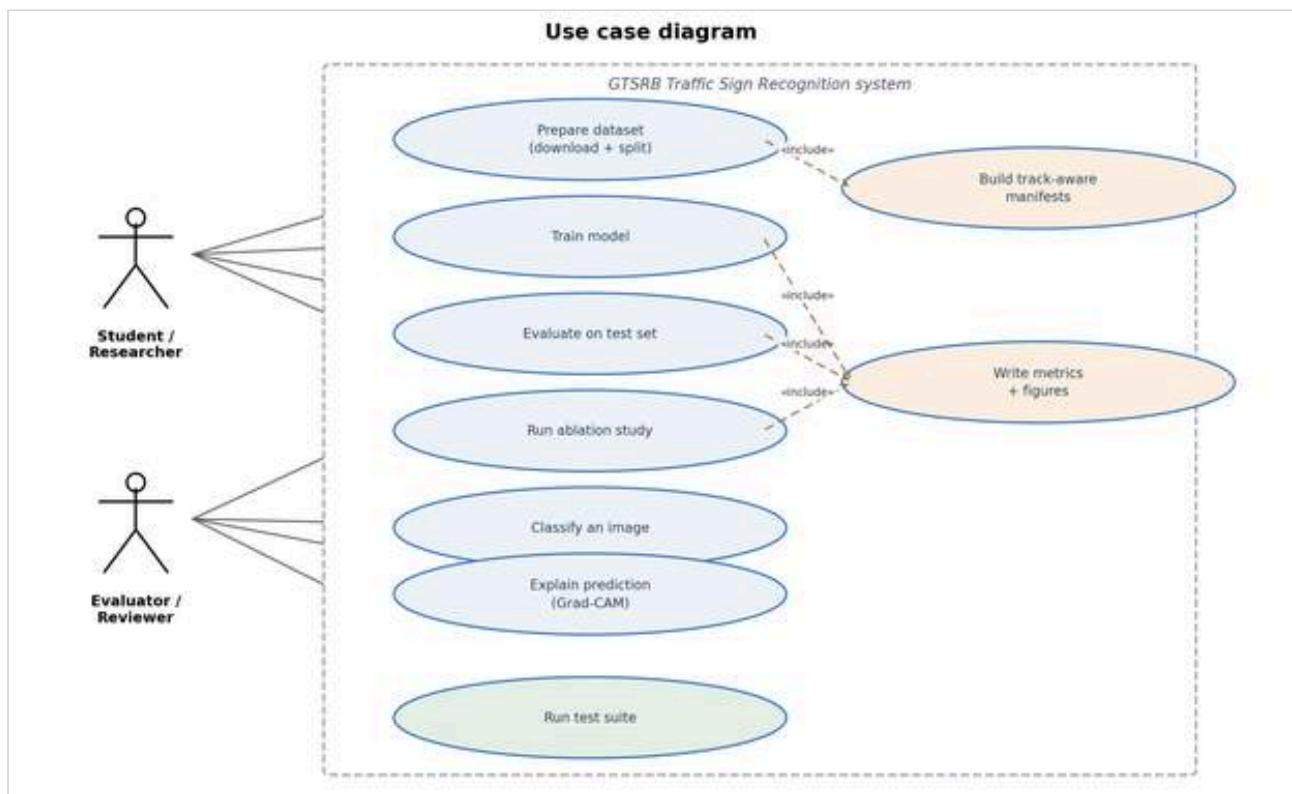


Figure 15. Use case diagram. The two actors are the researcher who runs the pipeline and the reviewer who verifies it without retraining.

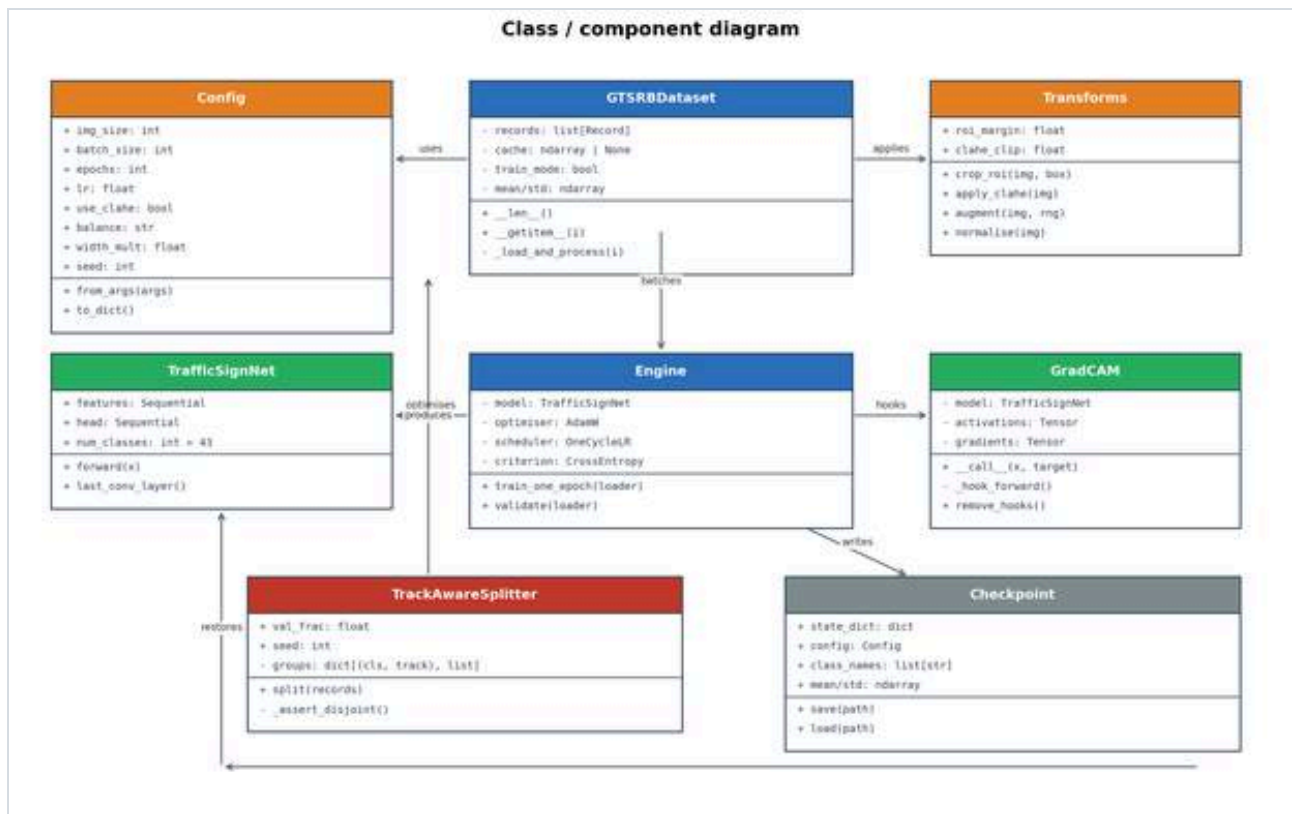


Figure 16. Class and component diagram, showing the types that carry state and the relations between them.

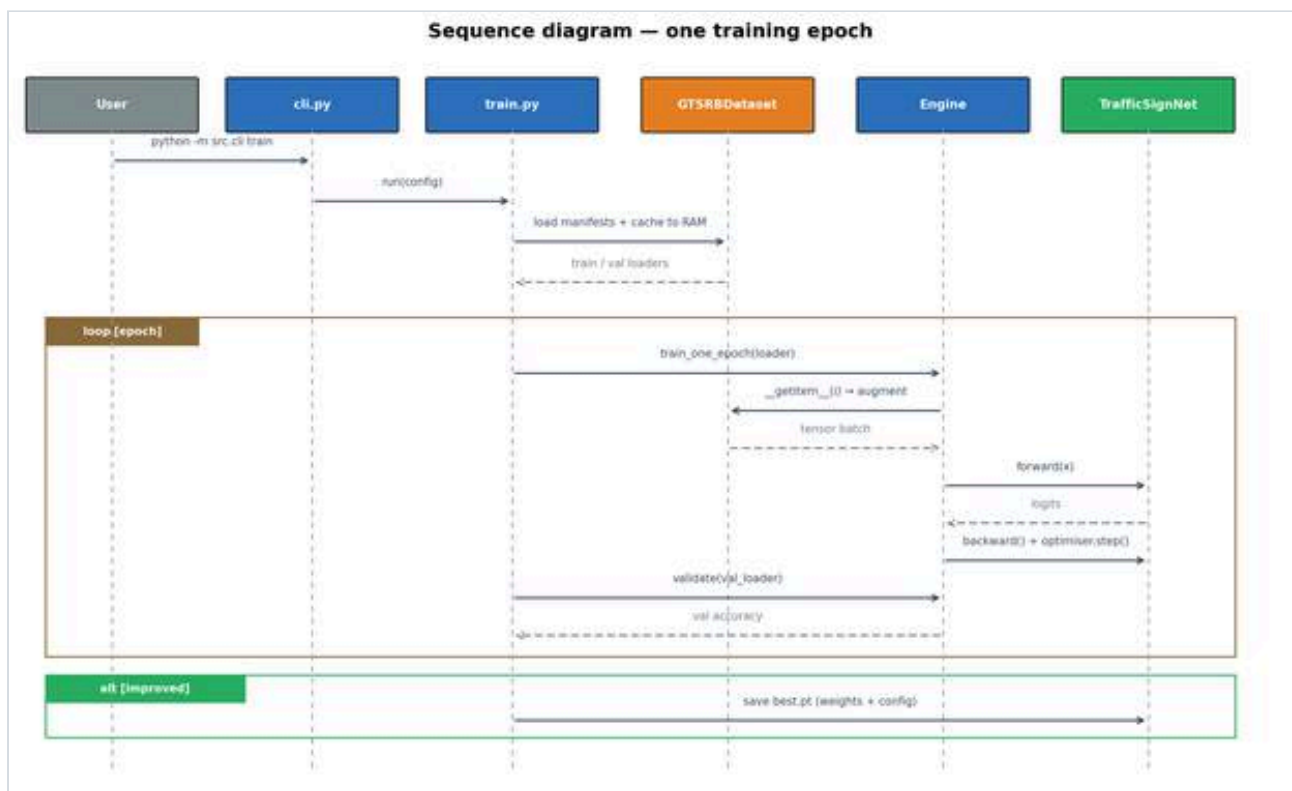


Figure 17. Sequence diagram for one training epoch, from the command line down to the optimiser step and the conditional checkpoint.

13. Design decisions and rationale

The decisions below are the ones where a different choice was plausible. Each is recorded with what was rejected and why, since a rationale is only meaningful against the alternative.

Decision	Alternative rejected	Rationale
Split by track	Random split by image	The decision the whole project rests on. Consecutive frames of one sign are near-duplicates, so a random split validates on memorised images and inflates the score toward 99.9%. Measured cost: a lower but truthful number.
Composite (class, track) key	Bare track number	Track numbering restarts at zero inside every class directory, so <code>00000</code> exists in several classes and refers to unrelated signs. A bare key silently merges them.
CLAHE on the LAB L-channel	Per-RGB-channel equalisation	Equalising channels independently shifts hue, and colour is class-discriminative for traffic signs. Worth 1.62 points (Section 9).
4×4 CLAHE tiles	OpenCV's 8×8 default	On a 32-pixel crop an 8×8 grid leaves ~4-pixel tiles, which amplifies noise rather than local contrast.
No flip augmentation	Horizontal flip, a standard default	Mirroring changes what a sign means: turn-left becomes turn-right, and speed-limit digits become unreadable. Actively wrong here, and enforced by a regression test.
One composed affine warp	Chained rotate, translate, scale	Three sequential warps interpolate three times and visibly smear a 32-pixel image. Composing into a single matrix interpolates once.
BORDER_REPLICATE	Zero-fill borders	Black corners correlate perfectly with "this sample was augmented", handing the network a shortcut feature.
Global average pooling head	Flatten → Linear	Drops ~88k parameters, makes the network accept any input size without code changes, and is the canonical arrangement for class activation maps.
bias=False in convolutions	Default bias	The BatchNorm immediately after re-centres the output, making a conv bias mathematically redundant.
Self-describing checkpoints	Weights only	Carrying the config, class names and normalisation statistics means <code>predict</code> and <code>evaluate</code> cannot be run with flags that silently mismatch the training run.
Figures generated from code	Exported images pasted in	A pasted figure drifts the moment the code changes and nobody notices. Every number in this report is read from <code>metrics.json</code> at build time.

14. Implementation and testing

The implementation is 15 modules under `src/`, each with a single responsibility, plus a test package. The layering in Figure 11 is enforced by convention rather than tooling: domain modules import from `config` and `utils` only, and no domain module imports an application module.

Testing approach. The suite is 50 unittest tests across five files, and its design follows one principle: *the claims a reviewer is most likely to doubt should be verifiable without the dataset*. The split logic is

therefore tested against synthetic records, so the central methodological claim runs on a bare clone in under a second.

Test file	What it establishes	Needs data?
<code>test_split_logic.py</code>	Whole tracks stay on one side; no image is lost or duplicated; every class appears on both sides; the split is deterministic given a seed. One test deliberately demonstrates the bug being avoided — that a random split <i>does</i> leak tracks.	no (synthetic)
<code>test_split_integrity.py</code>	The same invariants against the real manifests once <code>prepare</code> has run.	yes (skips)
<code>test_transforms.py</code>	CLAHE raises contrast while preserving hue; crops respect image bounds; augmentation is deterministic per seed, never mirrors, and leaves no black border.	no
<code>test_model.py</code>	Output shape, parameter budget, resolution independence, absence of conv bias, gradient flow; Grad-CAM map shape, normalisation, hook cleanup and correct behaviour in eval mode.	no
<code>test_samples.py</code>	The committed <code>.ppm</code> samples survive a git checkout byte-exact — a PPM has an ASCII header before binary pixels, so git can mistake it for text and rewrite line endings, silently altering pixel data.	no

Validation is also enforced at runtime rather than only in tests: the splitter asserts its own disjointness invariant before writing manifests, so a regression in the grouping logic fails immediately and loudly instead of producing a plausible-looking but leaking dataset.

15. Challenges and key learnings

The near-duplicate frames. The first working version of this project reported about 99.9% validation accuracy, and that number was the most useful mistake in the work. It was implausibly high, and investigating why led to the filename structure — `{track}_{frame}.ppm` — and the realisation that 39,209 images depict only 1,307 distinct physical signs. Rebuilding the split by track dropped the reported figure and made it mean something. The learning generalises past this dataset: when data is collected in bursts, the unit of independence is the burst, not the row, and an implausibly good score is evidence to investigate rather than to celebrate.

The track-numbering trap. Grouping by the bare track number looked correct and silently merged unrelated signs, because numbering restarts inside each class directory. This was caught by asserting an invariant — that no `(class, track)` pair appears on both sides — rather than by reading the code again. Encoding assumptions as assertions found a bug that inspection had already missed twice.

Augmentation that destroys labels. Horizontal flipping is such a standard default that it was in the pipeline before the sign semantics were considered: it turns turn-left into turn-right and makes speed-limit digits unreadable. The general lesson is that an augmentation is only valid if it preserves the label, and that is a property of the domain, not of the library.

Grad-CAM silently returning nothing. Two subtleties cost real time: the older `register_backward_hook` reports wrong gradients for multi-input modules, and running under `no_grad` produces an empty map with no error. The method needs `eval()` mode for BatchNorm statistics but gradients still enabled — a combination that is easy to get wrong because both halves look individually correct.

Working within a CPU budget. The constraint forced measurement rather than guesswork about capacity and resolution. Halving the model width cost 5.39 points, which showed the 99k-parameter baseline was not padded, and the ablations made the case for each preprocessing step empirically instead of by appeal to convention.

16. Limitations and future work

- **Detection is out of scope.** GTSRB supplies ground-truth regions of interest, so the model receives a correctly cropped sign. A deployed system must first locate signs in a full road scene, and detection errors would compound with classification errors. The accuracy reported here is an upper bound on what a complete pipeline would achieve.
- **Input resolution is the binding constraint on accuracy.** Section 8 traced most errors to digits and pictograms that do not survive downsampling to 32×32 . Training at 48×48 or 64×64 is the most direct improvement available; the cost is roughly double the training time per epoch, which the CPU-only budget did not permit here.
- **The explanation maps are coarse.** 8×8 Grad-CAM confirms the model attends to the sign but cannot resolve finer evidence.
- **Single dataset, single country.** All data is German. No claim is made about transfer to other sign systems, which differ in colour convention and pictogram design.
- **One trained run per configuration.** The ablation differences of 0.51 points and above are reported from single runs. The larger effects are unambiguous, but the smallest would be better supported by repeated runs with different seeds.

17. Reproducibility

The project runs entirely from the command line. Every result in this report is regenerated by three commands: `prepare` downloads and splits the data, `train` fits the model, and `evaluate` produces the metrics and figures. Random number generators for Python, NumPy and PyTorch are seeded, and augmentation draws from a per-index generator so results hold even when data loading is parallelised.

One caveat is worth stating honestly: run-to-run reproducibility on a single machine is reliable, but bit-exact equality across different machines, BLAS builds or PyTorch versions is not guaranteed, because a few CPU kernels have no deterministic implementation. A trained checkpoint is committed to the repository so the reported numbers can be verified without retraining.

The test suite contains 50 tests. Most run without the dataset present, including the synthetic checks of the splitting logic described in Section 3, so the central methodological claim can be verified on a fresh clone in under a second.

18. Conclusion

A three-block convolutional network with 99,019 parameters classifies German traffic signs at 96.82% accuracy on the official GTSRB test set, training in 20 minutes on a CPU and classifying an image in 0.67 ms. Ablations confirm that contrast equalisation, class rebalancing and the chosen model capacity each contribute measurably, and Grad-CAM indicates the network attends to signs rather than their surroundings.

The result that matters most, though, is methodological. Because GTSRB's images come in near-duplicate bursts of the same physical sign, an evaluation split drawn at random measures memorisation rather than generalisation. Splitting by track produces a lower number, and a truthful one. Residual errors concentrate in visually similar sign families and trace to input resolution, which points clearly at where further work should go.

19. References

1. J. Stallkamp, M. Schlipsing, J. Salmen, C. Igel. The German Traffic Sign Recognition Benchmark: A multi-class classification competition. *IJCNN*, 2011.
2. R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, D. Batra. Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization. *ICCV*, 2017.
3. S. Ioffe, C. Szegedy. Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. *ICML*, 2015.
4. L. N. Smith, N. Topin. Super-Convergence: Very Fast Training of Neural Networks Using Large Learning Rates. *arXiv:1708.07120*, 2018.
5. K. Zuiderveld. Contrast Limited Adaptive Histogram Equalization. *Graphics Gems IV*, 1994.
6. I. Loshchilov, F. Hutter. Decoupled Weight Decay Regularization. *ICLR*, 2019.

Generated from `results/metrics.json` by `python -m src.build_report`. Every figure and number in this report is produced by the committed code. Dataset courtesy of the Institut für Neuroinformatik, Ruhr-Universität Bochum.